Politecnico di Milano

Ingegneria del Software – a.a. 2004/05

I Prova - 2 Maggio 2005

Cognome
Nome
Matricola
Sezione (segnarne una) Baresi ?, Ghezzi ?, Morzenti ?, SanPietro ?

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare **solo i fogli distribuiti** utilizzando il **retro** delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. **non** verranno **in nessun caso** presi in considerazione. È possibile scrivere **in matita**.
3. È possibile consultare liberamente libri, manuali o appunti. È **proibito** l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. Tempo a disposizione: 2h15m.
6. Punteggio totale a disposizione: 14/30.

Esercizio 1: .....

Esercizio 2: .....

Esercizio 3: .....

Totale .....

Esercizio 1 (punti 4)

Si consideri il seguente programma Java:

```
public class ClasseA {
      protected static int num = 0;
      protected int val;

      public ClasseA(){
            val =0;
      }

      public void set(int a){
            num++;
            val = a;
      }

      public void stampa(){
            System.out.print("num: " + num);
            System.out.println(" val: " + val);
      }

      public static void main(String[] args) {
            ClasseA o1, o2;

            o1 = new ClasseA();
            o2 = new ClasseA();

            o1.set(3);
            o2.set(4);
            o1.stampa();
            o2.stampa();
}
```

Parte A

Cosa leggiamo sullo schermo dopo aver eseguito il programma?

num: 2 val: 3
num: 2 val: 4

Parte B

Si supponga poi di aggiungere le due sottoclassi e le dichiarazioni delle variabili a, b e c che seguono

```
public class ClasseB extends ClasseA {…}
public class ClasseC extends ClasseA {…}


ClasseA a;
ClasseB b;
ClasseC c;
```

Quali delle seguenti righe di codice darebbe un errore in compilazione? (mettere una X per ogni riga che si ritiene scorretta aggiungendo una breve spiegazione)

```
✍ a = new ClasseB();
✍ b = new ClasseA(); // Non si può assegnare ad una variabile un oggetto di una
                     // sua super classe
✍ c = new ClasseC();
✍ b = a; // come sopra
✍ b = new ClasseB();
✍ a = b;
✍ c = b; // Il tipo di b non è sottotipo di quello di c, né viceversa
```

Parte C

Si supponga anche di aggiungere il metodo usaVal alla ClasseA:

```
public int usaVal(ClasseA a){
      return (val+a.val);
}
```

e di includere nelle classi ClasseB e ClasseC i seguenti frammenti di codice:

```
public class ClasseB extends ClasseA {
…
      public int usaVal(ClasseB b){
            return (val*b.val);
      }
}

public class ClasseC extends ClasseA {
…
      public int usaVal(ClasseA c){
            return (val-c.val);
      }
}
```

Indicare, nei due casi della ClasseB e della classeC, se si tratta di ridefinizione del metodo usaVal o di overloading.

Si tratta di overloading nel primo caso, di ridefinizione nel secondo.

Si immagini che venga eseguito il seguente frammento di codice come parte del main della classe ClasseA.

```
ClasseA a = new ClasseA();
ClasseB b = new ClasseB();
ClasseC c = new ClasseC();

a.set(2); b.set(4); c.set(8);

System.out.println("risultato = " + a.usaVal(a));
System.out.println("risultato = " + a.usaVal(b));
System.out.println("risultato = " + b.usaVal(c));
System.out.println("risultato = " + c.usaVal(b));
```

Indicare, per ogni invocazione del metodo usaVal nelle istruzioni println, quale è la classe in cui è definito il metodo usaVal eseguito, fornendo una breve giustificazione della risposta.

Vengono eseguiti i metodi definiti, rispettivamente, in ClasseA (unico metodo disponibile per a), ClasseA (idem c.s.), ClasseA (parametro attuale *c* fatto corrispondere al parametro formale di tipo ClasseA dell'usaVal di ClasseA, non può corrispondere al parametro formale di tipo ClasseB dell'usaVal di ClasseB), ClasseC (parametro attuale b fatto corrispondere al parametro formale di tipo ClasseA dell'usaVal di ClasseC).

Riportare il risultato dell’esecuzione delle istruzioni println.

```
risultato = 4
risultato = 6
risultato = 12
risultato = 4
```

Esercizio 2 (punti 4)

Si consideri la seguente specifica informale di un'astrazione procedurale.

```
//@ ensures (* restituisce true sse parola compare almeno una volta in testo*)
public static boolean sottoStringa(char[] testo, char[] parola)
```

Esempi: testo = [abbcccddbccc] e parola = [bccc], risultato è true
Se testo = [abbcccddcc] e parola = [bccd], risultato è false

Parte A

Si migliori la specifica aggiungendo opportune precondizioni JML sui due array affinché l'operazione sottoStringa sia sensata.

```
//@ requires ..... testo!= null && parola != null;
//@ ensures (* restituisce true sse parola compare almeno una volta in testo*)
public static boolean sottoStringa(char[] testo, char[] parola)
```

E' invece inutile aggiungere la precond. che *parola* sia più corta di *testo*, in quanto se ciò accadesse *parola* non sarebbe una sottostringa.

Parte B

Si trasformino le precondizioni di cui al punto (a) nel lancio di opportune eccezioni, specificandole in JML.

```
//@ ensures .....testo!= null && parola != null
//@ && (* restituisce true sse parola compare almeno una volta in testo*);
//@ signals ..... (NullPointerException e) testo ==null || parola == null;
public static boolean sottoStringa(char[] testo, char[] parola) .....
throws NullPointerException
```

Parte C

Si riscriva la clausola ensures utilizzando JML, in modo completamente formale (senza commenti) facendo sì che la postcondizione risulti *eseguibile*.

```
//@ ensures .....testo!= null && parola != null && \result <==>
//@ ..... (\exists int i; 0<=i && i <=testo.length-parola.length;
//@ ..... (\forall int j; 0<=j && j<parola.length; testo[i+j] == parola[j]));
public static boolean sottoStringa(char[] testo, char[] parola) .....
```

Esercizio 3 (punti 6)

Si consideri una classe *SetterGetter*, i cui esemplari rappresentano multi-insiemi di oggetti di una classe astratta *ObjectWithKind*; questa sostanzialmente aggiunge a Object un campo costante "kind" intero positivo, che specifica il "tipo" degli oggetti (abbiamo così, per esempio, oggetti di tipo 7).

```
abstract class ObjectWithKind extends Object {
      public final int kind;
      //le implementazioni di questa classe garantiscono che kind
      // sia >0; inoltre il suo valore non è modificabile
}
```

Gli oggetti di classe SetterGetter sono multi-insiemi nel senso che possono contenere diversi oggetti di classe ObjectWithKind aventi lo stesso valore dell'attributo kind; tuttavia i multi-insiemi non ammettono ripetizioni dello stesso oggetto e pertanto ogni oggetto può comparire al più una sola volta in ogni multi-insieme.
La classe SetterGetter fornisce le seguenti operazioni:

1. costruttore di default, che inizializza a vuoto il multi-insieme;
2. operazione observer boolean isIn(ObjectWithKind o) che restituisce true se e solo se l'oggetto o è presente nel multi-insieme;
3. operazione observer int curKind() che dà il tipo di oggetto da estrarre dal multi-insieme, come impostato dal più recente startKind(); se non vi è stata alcuna invocazione di startKind() viene restituito il valore 0;
4. operazione observer int howMany(int i) che restituisce il numero di elementi di tipo i presenti nel multi-insieme;
5. operazione startKind(x), con parametro intero positivo, che fa sì che da quel momento in avanti (fino a un'eventuale successiva startKind()) l'operazione get() prelevi un elemento il cui kind vale x eliminandolo dal multi-insieme.
6. operazione add(o) che inserisce nel multi-insieme un ObjectWithKind o;
7. operazione get() che preleva un elemento dal multi-insieme (vedi anche i punti precedenti); se non è possibile prelevare alcun elemento viene lanciata un'eccezione NoObjectWithKindException. Nel caso in cui esista più di un elemento che possa essere prelevato, non viene precisato quale debba essere scelto. Non è possibile prelevare alcun elemento prima che sia stata effettuata almeno un'invocazione del metodo startKind().

Parte A

Si fornisca una specifica della classe SetterGetter sulla base della descrizione a parole fornita qui sopra. I metodi osservatori sono già specificati in modo informale mediante commenti JML. Si aggiungano le specifiche completamente formalizzate per tutti gli altri metodi.

```
public class SetterGetter {

// observers
//@ ensures (* \result è true se e solo se o è incluso in this *);
public /*@ pure @*/ boolean isIn(ObjectWithKind o){…}

//@ ensures (* \result restituisce il kind corrente, cioè quello definito
//@ dall'ultima startKind, oppure 0*);
public /*@ pure @*/ int curKind(){…}

//@ ensures (* \result è il numero degli oggetti in this con kind k *);
public /*@ pure @*/ int howMany(int k){…}

//@ ensures this.curKind() == 0 && (\forall int i; i>0; this.howMany(i) == 0) &&
//@          (\forall ObjectWithKind o; ; !this.isIn(o));
public SetterGetter(){…}

//@ requires x > 0;
//@ ensures this.curKind() == x && this.howMany(x) > 0;
//@
```

```
//@ signals (NoObjectWithKindException e) this.howMany(x) == 0;
public void startKind(int x) throws NoObjectWithKindException {…}

//@ requires o != null;
//@ ensures this.isIn(o) &&
//@(\forall ObjectWithKind x; x != o;this.isIn(x)<==> \old(this.isIn(x)));
public void add(ObjectWithKind o){…}

//@ ensures this.curKind()>0 &&\old(this.howMany(this.curKind()))>0 &&
//@           \result.kind == this.curKind() &&
//@          !this.isIn(\result) && (\exists int i; \old(isIn(i)); \result==i))
//@          (\exists ObjectWithKind o; \old(this.isIn(o)); o == \result) &&
//@          (\forall ObjectWithKind o; o != \result; this.isIn(o) <==>
//@                   \old(this.isIn(o)));
//@ signals (NoObjectWithKindException e)  this.curKind()==0 || this.curKind()>0 &&
//@              \old(this.howMany(this.curKind()))==0;
public ObjectWithKind get() throws NoObjectWithKindException {…}
}
```

Parte B

Con riferimento a un'implementazione della classe SetterGetter basata sulle seguenti linee:

- un vector mi memorizza gli elementi presenti nel multi-insieme, disposti in un ordine qualunque, non precisato, che presumibilmente dipende dalla successione degli inserimenti e delle estrazioni;
- una variabile intera kind memorizza il valore corrente del tipo di elemento di cui si possono prelevare elementi; se non è definito alcun tipo di elemento da prelevare (non è stato mai invocato il metodo startKind) allora kind = 0;
- una variabile intera current individua nel vettore mi la posizione dell'ObjectWithKind prossimo da prelevare (il cui valore dell'attributo kind coincide con this.kind); se non è definito alcun tipo di elemento da prelevare (this.kind vale 0) essa può avere qualsiasi valore, mentre se il tipo di elemento da prelevare è definito (quindi this.kind!=0) ma non vi sono nel multi-insieme elementi di quel tipo (per esempio, perché sono stati tutti prelevati) current ha un valore <0 oppure >=mi.size().

Si definisca in JML il rep invariant e si implementi il metodo get.

```
public class SetterGetter {
      private Vector mi;
      private int kind;
      private int current;

/*@ private invariant mi != null && kind >= 0 &&
 (\forall int i; 0<=i && i<mi.size(); mi.get(i) instanceof ObjectWithKind &&
  this.kind>0 ==>
       (0<=current && current <mi.size()==>((ObjectWithKind) mi.get(current)).kind
== this.kind) &&
       !(0<=current && current <mi.size()==>
       !(\exists int i; 0=<i && i<mi.size(); ((ObjectWithKind) mi.get(i)).kind ==
this.kind)
@*/
```

L'ulteriore vincolo che tutti gli oggetti siano distinti (ad.es. (\forall int i; 0<=i && i<mi.size(); (\forall int j; i<j && j<mi.size(); mi.get(i) != mi.get(j))) non è previsto in modo esplicito dalla descrizione a parole, ma e' comunque ragionevole.

```
public ObjectWithKind get() throws NoObjectWithKindException {
   if (kind == 0 || current<0 || current >= mi.size())
                    throw new NoObjectWithKindException();
   ObjectWithKind o = (ObjectWithKind) mi.get(current);
   mi.set(current, mi.lastElement());
   mi.remove(mi.size()-1);
   if (mi.size() == 0)
      current = -1;
   else
      for (current = 0; current < mi.size() &&
               ((ObjectWithKind) mi.get(current)).kind != kind; current++);
   return o;
```

```
}
```